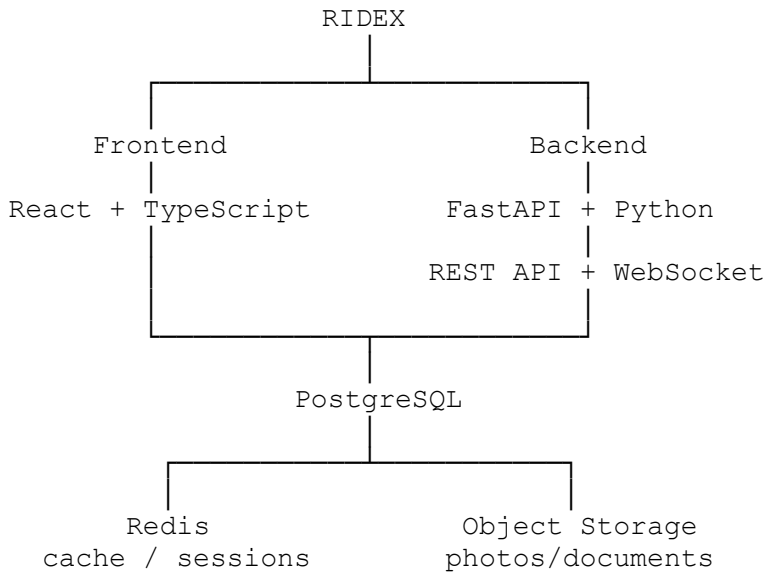
RIDEX – PROJECT

1. Technology Stack



Supporting technologies:

Frontend
————————

React
TypeScript
React Router
TanStack Query
Zustand
Tailwind CSS
Component system

Backend
————————

Python
FastAPI
Pydantic
SQLAlchemy
Alembic

Infrastructure
————————————

PostgreSQL
Redis
Docker
Nginx
Git/GitHub

Testing
————————

Pytest
Playwright
API testing

2. Architecture

```
backend/
│
├── app/
│   │
│   ├── core/
│   │   ├── config
│   │   ├── security
│   │   ├── database
│   │   └── exceptions
│   │
│   ├── modules/
│   │   │
│   │   ├── auth/
│   │   ├── users/
│   │   ├── vehicles/
│   │   ├── rides/
│   │   ├── bookings/
│   │   ├── payments/
│   │   ├── reviews/
│   │   ├── messaging/
│   │   ├── notifications/
│   │   ├── locations/
│   │   ├── safety/
│   │   └── admin/
│   │
│   ├── shared/
│   │
│   └── main.py
│
├── tests/
│
├── migrations/
│
├── Dockerfile
└── requirements.txt
```

3. Frontend Architecture

```
frontend/
└── src/
    │
    ├── app/
    │   ├── router/
    │   ├── providers/
    │   └── configuration/
    │
    ├── features/
    │   ├── auth/
    │   ├── rides/
    │   ├── bookings/
    │   ├── profile/
    │   ├── vehicles/
    │   ├── messages/
    │   ├── notifications/
    │   └── reviews/
    │
    ├── components/
    │   ├── ui/
    │   └── layout/
    │
    ├── pages/
    │
    ├── services/
    │
    ├── hooks/
    │
    ├── lib/
    │
    └── types/
```

4. RideX Feature Universe

Account & Identity

Users should be able to:

- Register
- Login/logout
- Email verification
- Phone verification
- Profile
- Profile picture
- Name
- Bio
- Gender preference
- Languages
- Member since
- Ratings
- Completed rides
- Driver/passenger statistics
- Account settings
- Privacy settings
- Notification preferences

5. Driver Module

```
Create Ride
    ↓
Define route
    ↓
Choose departure
    ↓
Choose arrival
    ↓
Date/time
    ↓
Available seats
    ↓
Price per passenger
    ↓
Preferences
    ↓
Publish
```

Ride preferences could include:

- Smoking / non-smoking
- Pets
- Luggage
- Music
- Conversation preference
- Maximum detour
- Gender preference where legally/appropriately applicable
- Instant booking vs approval

Then:

Driver Dashboard

Upcoming rides

Active rides

Completed rides

Bookings

Earnings

Reviews

Cancellation history

6. Passenger Module

Passenger gets:

SEARCH

From: Chennai
To: Bangalore
Date: September 12
Passengers: 1

[Search]

Then highly optimized results:

Chennai ———— *Bangalore*

07:00 AM 12:30 PM

₹450 / seat
★ 4.9 Rahul
Swift
2 seats available

[View ride

Filters:

- Departure time
- Price
- Seats
- Driver rating
- Vehicle
- Instant booking
- Preferences
- Pickup/drop points
- Detour
- Route

7. Booking System

This should be a proper business module, not simply:

booking = Booking(...)

We need states.

```
REQUESTED
    ↓
CONFIRMED
    ↓
UPCOMING
    ↓
IN_PROGRESS
    ↓
COMPLETED
```

Alternative paths:

```
REQUESTED → REJECTED
REQUESTED → CANCELLED
CONFIRMED → CANCELLED
IN_PROGRESS → COMPLETED
```

This becomes extremely important later for payments, refunds, notifications and analytics.

8. Payments

Eventually:

```
Passenger
    ↓
Booking
    ↓
Payment
    ↓
Confirmation
    ↓
Ride completed
    ↓
Driver settlement
```

We shouldn't hard-wire the application to one payment provider.

Instead:

```
PaymentService
        |
        ├── RazorpayProvider
        ├── StripeProvider
        └── MockPaymentProvider
```

9. Maps & Locations

A ride-sharing platform becomes much better with location intelligence.

Features:

- Location autocompletes
- Route calculation
- Pickup points
- Drop points
- Distance
- Estimated travel time
- Route preview
- Nearby rides

Architecture:

```
RideX
 |
 └── LocationService
        |
        ├── Geocoding provider
        ├── Routing provider
        └── Map provider
```

10. Messaging

Eventually:

Passenger ↔ Driver

Features:

- Booking-related chat
- Messages
- Read/unread status
- Conversation list
- Notifications
- Message timestamps
- Basic moderation/reporting

This is where **WebSockets** become useful.

11. Reviews & Reputation

After completed rides:

Rate your experience

★★★★★

Cleanliness ★★★★★

Punctuality ★★★★☆

Communication ★★★★★

Overall ★★★★★

Comment:

"Great driver and very punctual."

12. Safety

This is something I would make a major part of the design.

Possible features:

```
Verified profiles
Emergency contact
Report user
Block user
Report ride
Trip sharing
Safety center
Ride check-in
Suspicious activity detection
```

For example:

```
Share Trip

Your trip:
Chennai → Bangalore

Share live trip with:
[ Mother ]
[ Friend ]
[ + Add contact ]

[Share]
```

13. Notifications

Instead of scattering notifications throughout the code:

```
NotificationService
        |
        ├── Email
        ├── Push
        └── In-app
```

Events could be:

```
BookingRequested
BookingConfirmed
BookingCancelled
RideStartingSoon
MessageReceived
PaymentSuccessful
ReviewRequested
```

Then multiple things can respond to the same event.

14. Admin Platform

A serious marketplace needs administration.

```
Admin Dashboard
──────────────────────────────

Users          18,492
Active rides    2,341
Bookings        7,829
Revenue         ₹XX,XXX
Reports            43
Suspended users    17
```

Admin capabilities:

- User management
- Ride management
- Booking management
- Reports
- Complaints
- Verification
- Suspensions
- Fraud monitoring
- Analytics
- Platform configuration

15. Search Performance

So performance will be a requirement **from the architecture stage**, not something we think about after the project is finished.

For example:

```
User searches:
Chennai → Bangalore
```

We shouldn't load every ride into React and filter them there.

Instead:

```
React
   ↓
GET /rides?from=...&to=...&date=...
   ↓
FastAPI
   ↓
PostgreSQL indexed query
   ↓
Only relevant rides
```

We'll use:

- Database indexes
- Pagination
- Filtering at database level
- Efficient joins
- Query optimization
- API response models
- Caching where appropriate
- Lazy loading
- Frontend request caching
- Debounced search
- Optimistic UI where appropriate

16. API Design

We should establish API conventions early.

For example:

```
/api/v1/auth
/api/v1/users
/api/v1/rides
/api/v1/bookings
/api/v1/reviews
/api/v1/vehicles
/api/v1/payments
```

17. Database Design

We'll design the database carefully rather than creating tables whenever we need one.

A rough starting point:

```
users
profiles
vehicles

rides
ride_stops
ride_preferences

bookings
payments

reviews
messages

conversations

notifications

locations

reports
support_tickets
```

And we'll use migrations.

```
Migration 001
   ↓
Migration 002
   ↓
Migration 003
   ↓
...
```

So, a future database modification is reproducible and safe.

18. Design System

I don't want every page designed independently.

We'll build a design system first:

```
Design System
│
├── Colors
├── Typography
├── Spacing
├── Buttons
├── Inputs
├── Cards
├── Modals
├── Dropdowns
├── Navigation
├── Tables
├── Notifications
├── Loading states
└── Empty states
```

Then:

```
Ride Card
Booking Card
Profile Card
Vehicle Card
Review Card
```

all follow the same design language.

That makes the product feel like **one real application**, rather than a collection of pages.

19. Responsive Design

We shouldn't design it as:

"Desktop first, mobile later."

The component system should support:

```
Desktop
    ↓
Tablet
    ↓
Mobile
```

A ride-search card that looks like:

```
┌──────────────────────────────┐
│ Chennai → Bangalore          │
│                              │
│ 7:00 AM          ₹450        │
│                              │
│ Rahul      ⭐ 4.9            │
│                              │
│              [Book Ride]     │
└──────────────────────────────┘
```

should intelligently transform on mobile.

20. Development Method.

```
"Let's start coding."
```

We'll follow something closer to a real software-development workflow.

Phase 1 — Product Definition

Define:

```
Problem
Users
User journeys
Functional requirements
Non-functional requirements
Business rules
Constraints
```

Phase 2 — System Design

Define:

```
Architecture
Database
API contracts
Authentication
Authorization
Frontend structure
Error handling
Security model
```

Phase 3 — Design

Create:

```
Design system
Wireframes
Page hierarchy
User flows
Responsive layouts
Component specifications
```

Phase 4 — Development

Build module-by-module:

```
Authentication
       ↓
Profiles
       ↓
Vehicles
       ↓
Ride creation
```

```
        ↓
Ride search
        ↓
Booking
        ↓
Reviews
        ↓
Notifications
        ↓
Payments
        ↓
Messaging
        ↓
Safety
        ↓
Admin
```

Phase 5 — Testing

Every module gets:

```
Unit tests
API tests
Integration tests
Frontend tests
End-to-end tests
```

Phase 6 — Optimization

Measure:

```
API response time
Database query performance
Frontend rendering
Bundle size
Network requests
```

Then optimize based on evidence rather than guessing.

Phase 7 — Deployment

Eventually:

```
GitHub
   ↓
CI/CD
   ↓
Docker
   ↓
Production
```

21. The Golden Rule for This Project

I want us to follow this principle throughout:

Do not build today's feature in a way that makes tomorrow's feature difficult.

For example, don't write:

```
BookingService
```

that assumes only one payment method forever.

Instead:

```
Booking
   │
   └── Payment abstraction
           │
           ├── Mock
           ├── Razorpay
           └── Future provider
```

Similarly:

```
Notification
     │
     ├── In-app
     ├── Email
     └── Push
```

and:

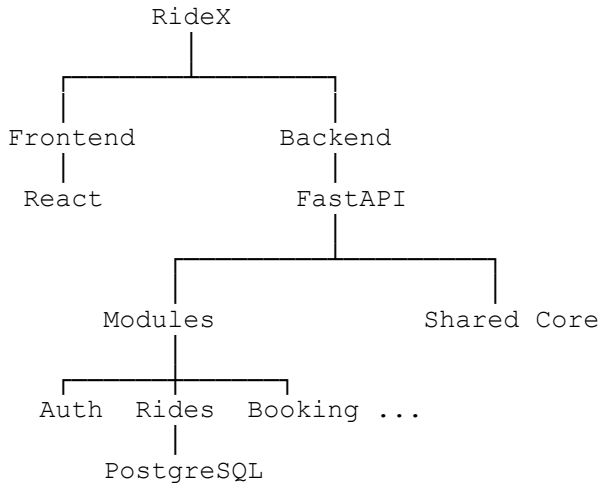
```
Location
     │
     ├── Geocoding
     └── Routing
```

This is what will make the project genuinely **future-proof**.

22. But We Shouldn't Overengineer It

We'll start with a:

Modular Monolith



Later, if RideX becomes large enough, individual modules can be extracted into services.

```
20 microservices
Kubernetes
Kafka
Redis Cluster
GraphQL
Event sourcing
CQRS
Machine learning
...
```

23. Our Product Roadmap

I'd divide the project into releases.

v0.1 — Foundation

```
Authentication
User profiles
Database
API architecture
Design system
```

v0.2 — Ride Marketplace

```
Create ride
Search rides
Ride details
Filters
Locations
```

v0.3 — Booking

```
Booking requests
Approval
Cancellation
Seat management
Booking history
```

v0.4 — Trust

```
Reviews
Ratings
Verification
Reports
Safety features
```

v0.5 — Communication

Notifications
Driver/passenger messaging
Email notifications

v0.6 — Payments

Payment abstraction
Mock payment
Real payment integration
Refund/cancellation logic

v0.7 — Intelligence

Better search
Recommendations
Price suggestions
Route optimization
Fraud/risk signals

v1.0 — Production Candidate

Security hardening
Performance optimization
Testing
Monitoring
CI/CD
Deployment
Documentation